

EMBEDDED SYSTEM WITH IOT EXPERIMENT NO. 6

IoT based Building System

Name: NAVARRO, ROD GERYK C.

Course/Section: CPE162P-4/E01

Date of Performance: 06/02/2025

Date of Submission: 06/06/2025

CYREL O. MANLISES, PH.D.

Instructor

DISCUSSION

Connections

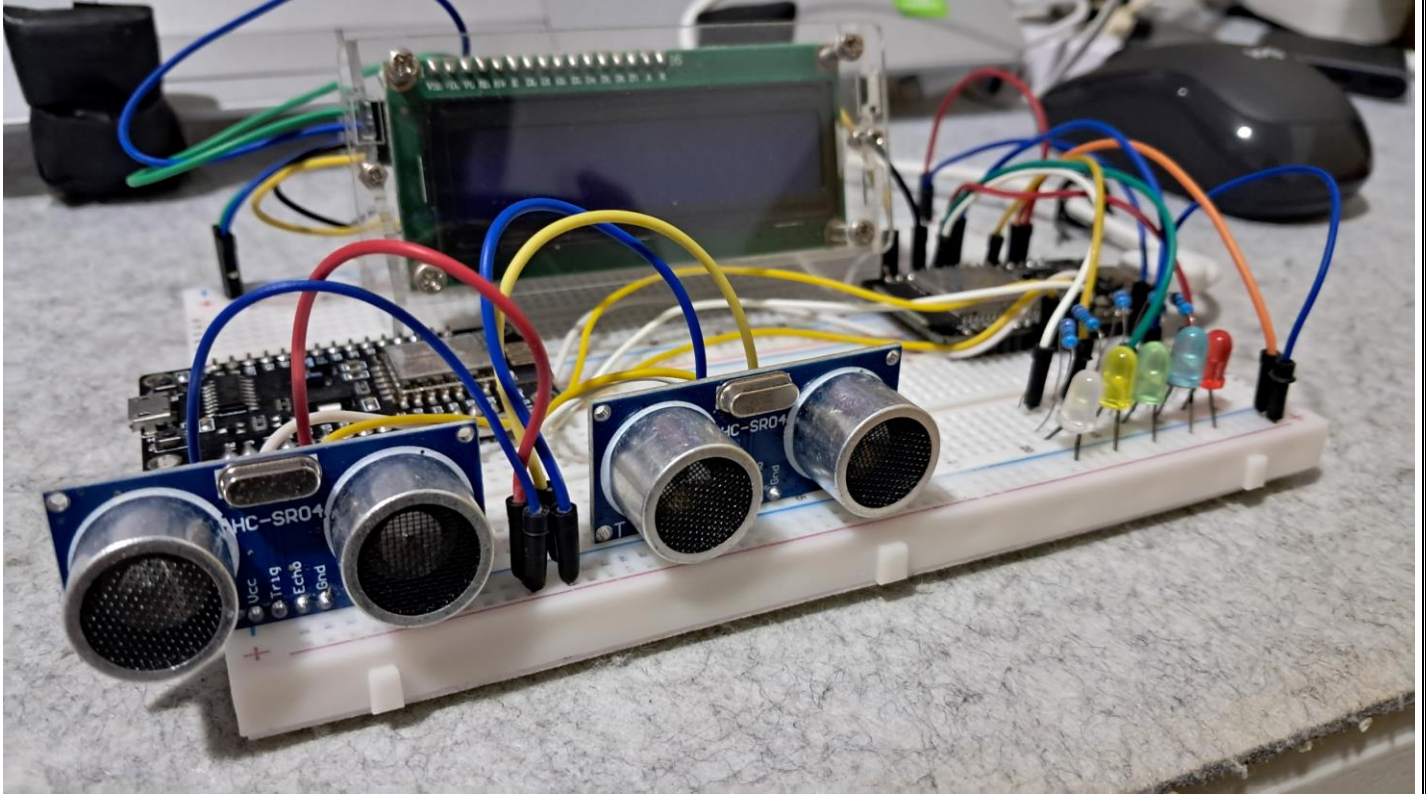


FIGURE 1: The Connection of the Ultrasonic Sensors, LEDs, and LCD to the ESP32 Dev Module

This project aims to control the lighting of a building floor based on how many people are inside, helping save electricity. It uses two ultrasonic sensors to count people entering and exiting. Based on the total number of people, a set of five LEDs is turned on or off to match the lighting needs. The system also displays the number of people and lighting status on an LCD and sends the data to ThingSpeak for cloud monitoring.

Code Development

```
Navarro_EXP6_CPE162P.ino
1  #include <LiquidCrystal_I2C.h>
2  #include <WiFi.h>
3  #include <ThingSpeak.h>
4
5  // WiFi Configuration
6  const char* ssid = "PLDTHOMEFIBRT6ucX";
7  const char* password = "PLDTWIFITLK72";
8
9  // ThingSpeak Configuration
10 const long myChannelNumber = 2964917;
11 const char myWriteAPIKey[] = "18A6ZBU4I3OLZZ44";
12
13 // Hardware Configuration
14 const int ENTRANCE_TRIG = 27;
15 const int ENTRANCE_ECHO = 26;
16 const int EXIT_TRIG = 25;
17 const int EXIT_ECHO = 33;
18 const int LED_PINS[] = {19, 18, 17, 16, 4}; // 5 LEDs
19
20 // System Parameters
21 const int UPDATE_INTERVAL = 15000;
22 const int MAX_PEOPLE = 50;
23 const int DISTANCE_THRESHOLD = 10; // cm
24 const int DEBOUNCE_DELAY = 500; // ms
25
26 // Global Variables
27 LiquidCrystal_I2C lcd(0x27, 16, 2);
28 WiFiClient client;
29 int peopleCount = 0;
30 unsigned long lastUpdate = 0;
31 unsigned long lastEntranceTrigger = 0;
32 unsigned long lastExitTrigger = 0;
33
34
35 void setup() {
36     // Initialize hardware
37     initializePins();
```

FIGURE 2: The Initialization of Variables and Components

In the beginning of the code, I included important libraries like LiquidCrystal_I2C, WiFi, and ThingSpeak. Then, I declared all the necessary variables such as the ultrasonic sensor pins, LED pins, and WiFi credentials. Arrays and constants like LED_PINS[], UPDATE_INTERVAL, and MAX_PEOPLE were used to make the code cleaner and easier to adjust. These variables will help track the number of people and manage the system's behavior.

```

Navarro_EXP6_CPE162P.ino
32 // Initialize variables
33
34
35 void setup() {
36     // Initialize hardware
37     initializePins();
38     initializeDisplay();
39     initializeSerial();
40
41     // Connect to WiFi
42     connectToWiFi();
43     ThingSpeak.begin(client);
44 }
45
46 void loop() {
47     // Update people count
48     updatePeopleCount();
49
50     // Control lights based on occupancy
51     updateLighting();
52
53     // Update display
54     updateDisplay();
55
56     // Send data to cloud
57     sendToCloud();
58
59     delay(100);
60 }
61

```

FIGURE 3: The Setup of the Components

Inside the setup() function, I initialized the pins for sensors and LEDs, the LCD display, and the serial monitor for debugging. Then, I connected the ESP32 to WiFi so that it can send data to the cloud. The LCD shows status messages like "System Starting" and "WiFi Connected" to confirm everything is working during startup.

```

Navarro_EXP6_CPE162P.ino
46 void loop() {
47     // Update people count
48     updatePeopleCount();
49
50     // Control lights based on occupancy
51     updateLighting();
52
53     // Update display
54     updateDisplay();
55
56     // Send data to cloud
57     sendToCloud();
58
59     delay(100);
60 }
61

```

FIGURE 4: The Loop Function

The loop() function is where all the main actions happen continuously. It first updates the number of people by checking both entrance and exit sensors. Then, it decides how many lights should be turned on depending on the number of people inside. After that, it updates the LCD screen to show current data and finally sends this data to the ThingSpeak cloud. All of these steps are repeated again and again with a short delay.

```

Navarro_EXP6_CPE162P.ino
60 }
61
62 void initializePins() {
63     // Initialize ultrasonic sensors
64     pinMode(ENTRANCE_TRIG, OUTPUT);
65     pinMode(ENTRANCE_ECHO, INPUT);
66     pinMode(EXIT_TRIG, OUTPUT);
67     pinMode(EXIT_ECHO, INPUT);
68
69     // Initialize LEDs
70     for (int i = 0; i < 5; i++) {
71         pinMode(LED_PINS[i], OUTPUT);
72         digitalWrite(LED_PINS[i], LOW);
73     }
74 }
75

```

FIGURE 5: Initialize Pins Function

```

Navarro_EXP6_CPE162P.ino
79     }
80
81     void initializeDisplay() {
82         lcd.init();
83         lcd.backlight();
84         lcd.clear();
85         lcd.setCursor(0, 0);
86         lcd.print("System Starting");
87     }

```

FIGURE 6: Initialize Display Function

```

Navarro_EXP6_CPE162P.ino
200 int getActiveLights() {
201     if (peopleCount == 0) return 0;
202     if (peopleCount < 4) return 1;
203     if (peopleCount < 10) return 3;
204     if (peopleCount < 20) return 4;
205     return 5;
206 }
207
208 void sendToCloud() {
209     if (millis() - lastUpdate >= UPDATE_INTERVAL) {
210         lastUpdate = millis();
211
212         // Send data to ThingSpeak
213         ThingSpeak.setField(1, peopleCount); // people count
214         ThingSpeak.setField(2, getActiveLights()); // active lights
215
216         // Send individual LED status
217         for (int i = 0; i < 5; i++) {
218             ThingSpeak.setField(3 + i, digitalRead(LED_PINS[i])); // usage of
219         }
220
221         int status = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
222
223         if (status == 200) {
224             Serial.println("Cloud update successful");
225         } else {
226             Serial.println("Cloud update failed. Error: " + String(status));
227         }
228     }
229 }

```

FIGURE 7: Get Active Lights and Send to Cloud Function

```

Navarro_EXP6_CPE162P.ino
171 }
172 }
173
174 void updateDisplay() {
175
176 // Line 1: People count
177 lcd.setCursor(0, 0);
178 lcd.print("People: ");
179 lcd.print(peopleCount);
180
181 if (peopleCount < 10){
182 lcd.setCursor(0, 0);
183 lcd.print(" ");
184 }
185 // Line 2: Lights status
186 lcd.setCursor(0, 1);
187 lcd.print("Lit: ");
188
189 if (peopleCount == 0) lcd.print("OFF");
190 else if (peopleCount < 4) lcd.print("LOW");
191 else if (peopleCount < 10) lcd.print("MED");
192 else if (peopleCount < 20) lcd.print("HIGH");
193 else lcd.print("FULL");
194
195 lcd.print(" (");
196 lcd.print(getActiveLights());
197 lcd.print("/s) ");
198 }

```

FIGURE 8: Update Display Function

```

Navarro_EXP6_CPE162P.ino
86 lcd.print("System Starting ");
87 }
88
89 void connectToWiFi() {
90 lcd.clear();
91 lcd.print("Connecting WiFi");
92
93 WiFi.begin(ssid, password);
94 while(WiFi.status() != WL_CONNECTED) {
95 delay(500);
96 lcd.print(".");
97 }

```

FIGURE 9: Connect to WIFI Function

```

Navarro_EXP6_CPE162P.ino
115 }
116
117 void updatePeopleCount() {
118 int entranceDist = readUltrasonic(ENTRANCE_TRIG, ENTRANCE_ECHO);
119 int exitDist = readUltrasonic(EXIT_TRIG, EXIT_ECHO);
120
121 unsigned long currentTime = millis();

```

FIGURE 10: Update People Count Function

Several custom functions were created to keep the code organized. initializePins(), initializeDisplay(), and connectToWiFi() handle setup tasks. updatePeopleCount() increases or decreases the person count based on ultrasonic sensor readings. updateLighting() controls how many LEDs are on depending on how crowded the floor is. updateDisplay() shows the person count and lighting status on the LCD. Finally, sendToCloud() sends data like number of people, number of lights on, and each LED's status to ThingSpeak every 15 seconds.

Working Prototype

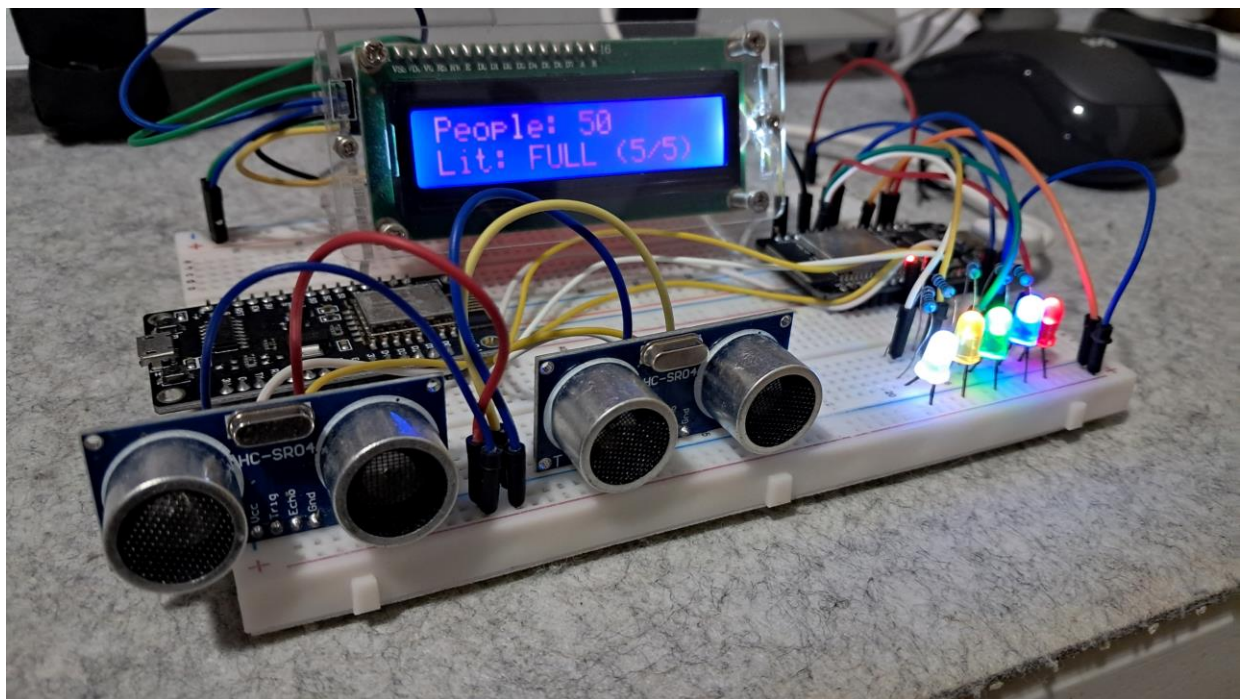
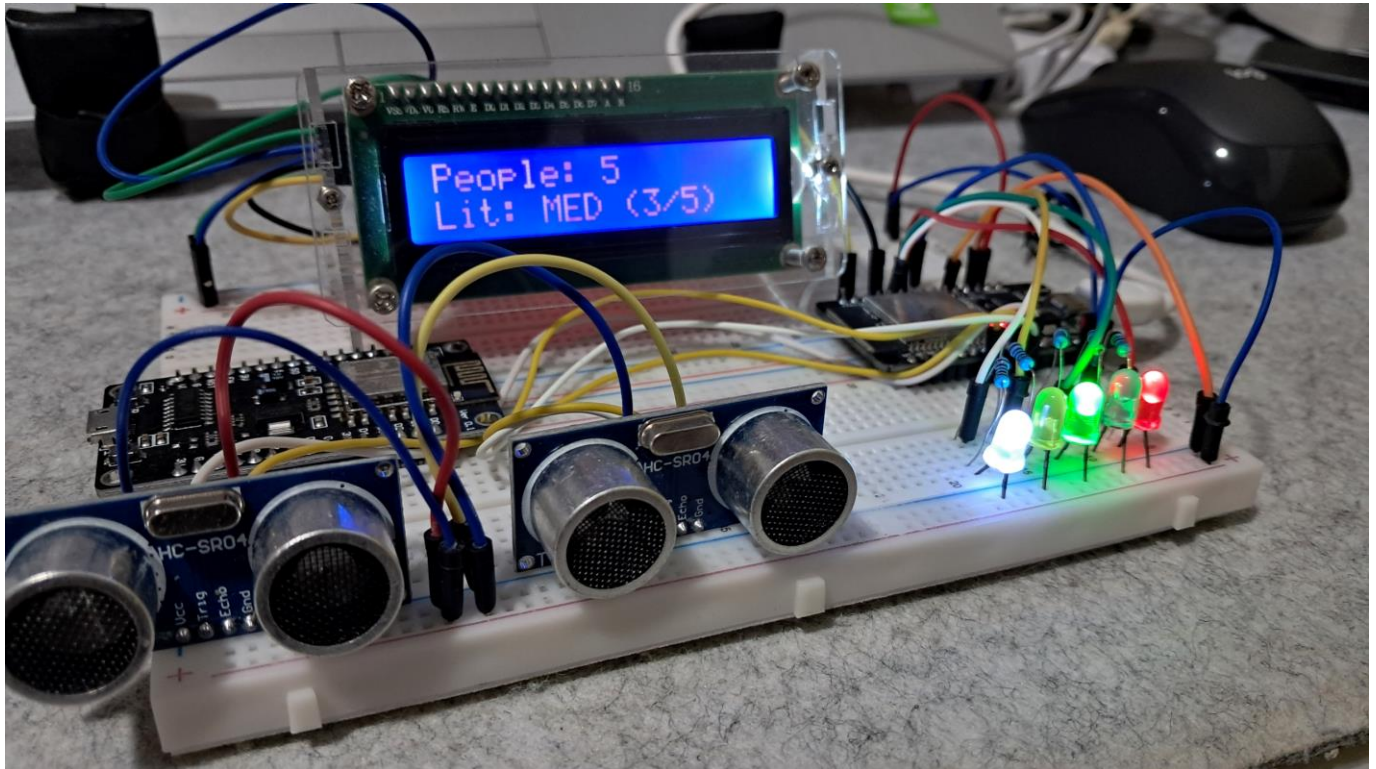


FIGURE 11: Working Prototype

This is the final working prototype of the IoT based building system experiment, which successfully meets all the requirements.

Interpretations of ThingSpeak's Graphs and Widgets



FIGURE 12: ThingSpeak Visualization: Low Energy Consumption

Since there are only two people inside the building, to save energy while giving enough energy for the people inside the building the system uses low light energy which is sufficient for the number of people inside. We can see on the LCD screen that there are only 2 people in the building floor with one LED turned ON. The graphs show the people count, energy consumption level, as well as the current status of each LEDs.



FIGURE 13: ThingSpeak Visualization: Medium Energy Consumption

More than four people are inside the building, we can see on the LCD screen that there are 7 people in the building floor with 3/5 LEDs turned ON. In the graphs we can notice that the increase of the number of people inside the building resulted to an increase in energy consumption and number of lights turned ON. Currently, the system is using a medium level of light energy for the people inside.

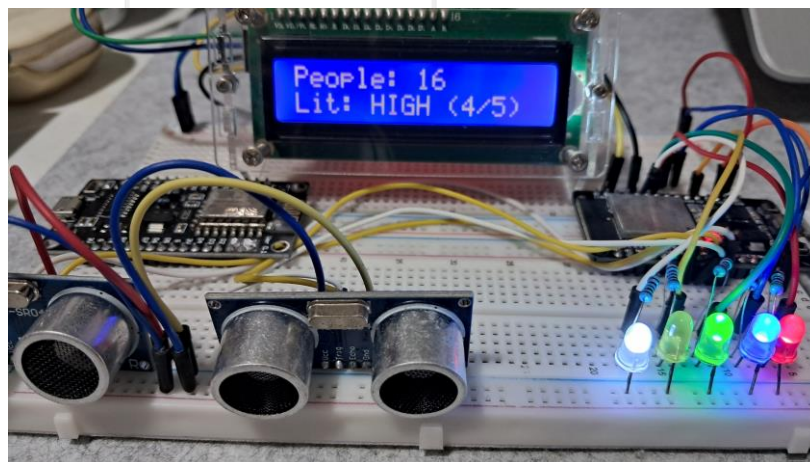


FIGURE 14: ThingSpeak Visualization: High Energy Consumption

The energy consumption level increases as the number of people in the building increases. There are more than ten people inside the building, We can see on the LCD screen that there are 16 people in the building floor with 4/5 LEDs turned ON. In the graphs we can notice that the increase of the number of people inside the building resulted to an increase in energy consumption and number of lights turned ON. Currently, the system is using a high level of light energy for the people inside.



FIGURE 15: ThingSpeak Visualization: Full Energy Consumption

Currently, the building already reached the maximum energy level with all lights are ON. There are more than twenty people inside the building, we can see on the LCD screen that there are 42 people in the building floor with 5/5 LEDs turned ON, which means that the building uses a lot of energy for the people inside the floor. In the graphs we can notice that the increase of the number of people inside the building resulted to an increase in energy consumption and number of lights turned ON. Currently, the system is using a max level of light energy for the people inside.

CONCLUSION

In this project, I was able to create an IoT-based building floor lighting system that helps save electricity by adjusting the number of lights based on how many people are inside. Using two ultrasonic sensors, the system can count how many people enter and exit the area. The number of people detected controls how many LEDs are turned on – fewer people means fewer lights, while more people results in more lights being used. This setup helps make sure energy is only used when needed. The information is also shown on an LCD display, including the current number of people and the lighting level.

The code played an important role in making the system work properly. I organized it using custom functions to keep everything clear and easy to update. I used libraries for the LCD, WiFi connection, and cloud platform (ThingSpeak), and I declared variables to store sensor pins and system settings. The setup() function prepared all components, while the loop() function kept checking the sensors, adjusting the lights, updating the display, and sending data to the cloud. I also added a debounce delay to avoid errors in counting people, and set an update interval so the data doesn't overload ThingSpeak.

Finally, the system was tested successfully, and the working prototype met all the goals of the experiment. It correctly detected the number of people and adjusted the LEDs accordingly. It also showed data on the LCD and uploaded useful information to ThingSpeak, where graphs and widgets visualized the energy usage. The visualizations clearly showed how lighting changes depending on how crowded the place is – from low to full energy consumption. This project helped me understand how hardware, sensors, and cloud services can work together to solve real-world problems like energy saving in buildings.